Algorithms for Optimization and Planning ▪ Roskilde University

Classical Planning

Factored states and heuristics for free

Jens Classen

Session 8 ▪ Russell & Norvig, Chapter 11

Today

1. Warm-up: three blocks
2. Describing planning problems: PDDL
3. Planning as search, forwards and backwards

   — *break* —
4. Heuristics for free: relaxing PDDL + lab
5. Other ways to plan, and beyond classical planning
6. Assignment 7: pre-marshalling in PDDL

By the end of today you can

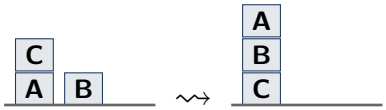
- write a planning problem in PDDL
- explain forward and backward search over factored states
- compute $h_{\max}$, h_{add} and h_{FF} by hand
- run a planner and read its output

Part 1

Warm-up: three blocks

Warm-up: stack the blocks

Pairs ▪ 6 min



Goal: `on(A,B)` **and** `on(B,C)`

Your turn: Plan by hand

A robot hand can hold one block. It can pick up a clear block (from the table or from another block), and put the block it holds on the table or on a clear block.

1. Find a shortest plan. How many steps?
2. Try the strategy "achieve one goal completely, then the other". Start with `on(A,B)`. Then try starting with `on(B,C)`. What happens?

The Sussman anomaly

- Shortest plan, 6 steps: unstack C from A, put C down, pick up B, stack B on C, pick up A, stack A on B.

The Sussman anomaly

- Shortest plan, 6 steps: unstack C from A, put C down, pick up B, stack B on C, pick up A, stack A on B.
- `on(A,B)` first: move C away, put A on B. Now B is covered: to get `on(B,C)` we must take A off again.
- `on(B,C)` first: put B on C. Now A is buried under C and B: undo again.

The Sussman anomaly

- Shortest plan, 6 steps: unstack C from A, put C down, pick up B, stack B on C, pick up A, stack A on B.
- `on(A,B)` first: move C away, put A on B. Now B is covered: to get `on(B,C)` we must take A off again.
- `on(B,C)` first: put B on C. Now A is buried under C and B: undo again.
- The two goals **interact**. Neither order of "one goal at a time" gives the shortest plan.

Gerald Sussman (1975) used this example to show the limits of planners that achieve goals one at a time.

What a planner needs to see

Which facts each action *needs*, which it *makes true*, and which it *destroys*.

In session 6, states were black boxes with a successor function. Today we look **inside** states and actions.

Part 2

Describing planning problems

From black boxes to facts

Search (session 6): a state is a black box. We only have

- $\textsc{Actions}(s)$ and $\textsc{Result}(s, a)$, written by hand,
- a goal test,
- a heuristic $h(s)$, invented by hand.

Planning: a state is a set of *facts*, and actions are described by *schemas*. From the description, the planner can work out

- which actions are applicable, and what they change,
- which actions are *relevant* for a goal,
- and a heuristic, *automatically*.

Why does that matter?

R&N's example: buy one particular book online. Every book number gives a "buy" action: billions of applicable actions in the start state.

A search algorithm tries them one by one. What would *you* do instead?

From black boxes to facts

Search (session 6): a state is a black box. We only have

- $\textsc{Actions}(s)$ and $\textsc{Result}(s, a)$, written by hand,
- a goal test,
- a heuristic $h(s)$, invented by hand.

Planning: a state is a set of *facts*, and actions are described by *schemas*. From the description, the planner can work out

- which actions are applicable, and what they change,
- which actions are *relevant* for a goal,
- and a heuristic, *automatically*.

Why does that matter?

R&N's example: buy one particular book online. Every book number gives a "buy" action: billions of applicable actions in the start state.

A search algorithm tries them one by one. What would *you* do instead?

Look at the goal $\texttt{have}(\texttt{book}_{42})$: only one action achieves it. A planner can do the same, because it can read the actions' effects.

States, goals and action schemas

- A **state** is a set of ground atoms, e.g. `{on(C,A), ontable(A), ontable(B), clear(B), clear(C), handempty}` Everything not in the set is false (closed world).
- A **goal** is a conjunction of atoms; it describes a *set* of states. `on(A,B)` $\wedge$ `on(B,C)` says nothing about the hand or the table.
- An **action schema** has parameters, a precondition and an effect:

STACK(x, y)	
PRE:	`holding(`x`)` $\wedge$ `clear(`y`)`
ADD:	`on(`x`,`y`), clear(`x`), handempty`
DEL:	`holding(`x`), clear(`y`)`

Semantics

A ground action a is **applicable** in s if $\text{PRE}(a) \subseteq s$. Then

$$\text{RESULT}(s, a) = (s \setminus \text{DEL}(a)) \cup \text{ADD}(a).$$

Everything the effect doesn't mention stays as it is.

A **plan** is a sequence of actions, each applicable in turn, that ends in a state satisfying the goal.

This is the STRIPS formalism (Fikes & Nilsson, 1971). R&N write the effect as one conjunction: DEL are the negated atoms.

PDDL: the domain file

```
(define (domain blocksworld)
  (:requirements :strips :typing)
  (:types block)
  (:predicates (on ?x - block ?y - block)
               (ontable ?x - block)
               (clear ?x - block)
               (handempty) (holding ?x - block))
  (:action pick-up
    :parameters (?x - block)
    :precondition (and (clear ?x) (ontable ?x)
                       (handempty))
    :effect (and (holding ?x)
                 (not (ontable ?x)) (not (clear ?x))
                 (not (handempty))))
  (:action stack
    :parameters (?x - block ?y - block)
    :precondition (and (holding ?x) (clear ?y))
    :effect (and (on ?x ?y) (clear ?x)
                 (handempty) (not (holding ?x))
                 (not (clear ?y))))
  ;; ... put-down and unstack in the same way
)
```

- **PDDL**, the Planning Domain Definition Language, is the standard input language of planners (McDermott et al., 1998, for the first planning competition).
- The **domain** file holds what is the same for all problems: types, predicates, action schemas.
- Variables start with ?. Lisp-style prefix notation.
- `(not ...)` in an effect is a delete.

PDDL: the problem file

```
(define (problem sussman)
  (:domain blocksworld)
  (:objects a b c - block)
  (:init (ontable a) (ontable b) (on c a)
         (clear b) (clear c) (handempty))
  (:goal (and (on a b) (on b c))))
```

- The **problem** file names the objects and gives the initial state (a complete list of true atoms) and the goal.
- A planner reads both files, **grounds** the schemas (all ways of filling in the parameters), and searches.

Quick questions

1. How many ground `stack` actions are there with 3 blocks? With 100 blocks?
2. Why is it useful to keep domain and problem apart?

PDDL: the problem file

```
(define (problem sussman)
  (:domain blocksworld)
  (:objects a b c - block)
  (:init (ontable a) (ontable b) (on c a)
         (clear b) (clear c) (handempty))
  (:goal (and (on a b) (on b c))))
```

- The **problem** file names the objects and gives the initial state (a complete list of true atoms) and the goal.
- A planner reads both files, **grounds** the schemas (all ways of filling in the parameters), and searches.

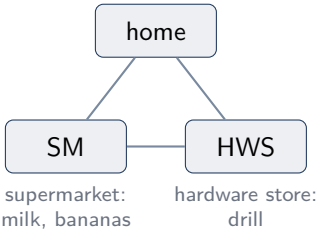
Quick questions

1. How many ground `stack` actions are there with 3 blocks? With 100 blocks?
2. Why is it useful to keep domain and problem apart?

(1) $3 \cdot 3 = 9$ (most are never applicable, e.g. `stack(a,a)`); 10 000. (2) One domain, many problems: the same model of the robot or the terminal for every task.

Your turn: the shopping trip in PDDL

Pairs, on paper ▪ 10 min



Start at home. Goal: have milk, bananas and a drill, and be back home.

Your turn: Write it in PDDL

1. Choose types and predicates. (Hint: three are enough.)
2. Write two action schemas: go from one place to another, and buy an item in a shop that sells it.
3. Write the problem file: objects, initial state, goal.
4. Find a shortest plan by hand. How long is it?

Check: does your go action make sure you can't be in two places at once?

Part 3

Planning as search

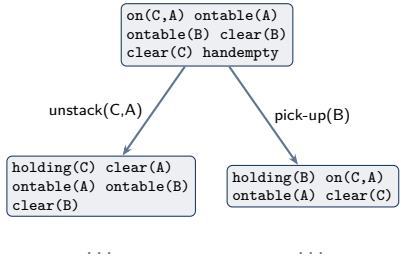
Forward (progression) search

The planning problem *is* a search problem:

- states: sets of ground atoms; start: `:init`
- successors: apply every applicable ground action
- goal test: all goal atoms are in the state
- cost: 1 per action (here)

So BFS, uniform-cost search, greedy search and A* from session 6 all work, unchanged.

But: many applicable actions are irrelevant, and the state space is huge: with n atoms there are up to 2^n states. Blind search is hopeless beyond toy problems.



Key idea

Forward search with a good *domain-independent* heuristic is how most of today's best planners work.

Backward (regression) search

Search from the goal back to the start, over goals (*sets* of states).

- A goal g has positive and negative literals, $\textsc{Pos}(g)$ and $\textsc{Neg}(g)$. Action a is **relevant** for g if it achieves a literal of g and undoes none (adds nothing in $\textsc{Neg}(g)$, deletes nothing in $\textsc{Pos}(g)$).
- **Regression** of g through a, kept separately:

$$\textsc{Pos}(g') = (\textsc{Pos}(g) \setminus \textsc{Add}(a)) \cup \textsc{Pos}(\textsc{Pre}(a))$$
$$\textsc{Neg}(g') = (\textsc{Neg}(g) \setminus \textsc{Del}(a)) \cup \textsc{Neg}(\textsc{Pre}(a))$$

- Stop when the start state satisfies g'.

In STRIPS, $\textsc{Neg}$ stays empty. The book example: one step back instead of billions forward.

Regress the shopping goal

$g =$ `have(drill), have(milk), have(bananas), at(home)`

1. Regress g through `buy(drill,hws)`.
2. Is there anything wrong with the result?

Backward (regression) search

Search from the goal back to the start, over goals (*sets* of states).

- A goal g has positive and negative literals, $\textsc{Pos}(g)$ and $\textsc{Neg}(g)$. Action a is **relevant** for g if it achieves a literal of g and undoes none (adds nothing in $\textsc{Neg}(g)$, deletes nothing in $\textsc{Pos}(g)$).
- **Regression** of g through a, kept separately:

$$\textsc{Pos}(g') = (\textsc{Pos}(g) \setminus \textsc{Add}(a)) \cup \textsc{Pos}(\textsc{Pre}(a))$$
$$\textsc{Neg}(g') = (\textsc{Neg}(g) \setminus \textsc{Del}(a)) \cup \textsc{Neg}(\textsc{Pre}(a))$$

- Stop when the start state satisfies g'.

In STRIPS, $\textsc{Neg}$ stays empty. The book example: one step back instead of billions forward.

Regress the shopping goal

$g =$ `have(drill), have(milk), have(bananas), at(home)`

1. Regress g through `buy(drill,hws)`.
2. Is there anything wrong with the result?

$g' =$ `have(milk), have(bananas), at(home), at(hws), sells(hws,drill)`: at home *and* at the hardware store. No state satisfies it, but regression doesn't notice. Such dead goals, and weaker heuristics, are why forward search is more popular today.

Break: 15 minutes

Puzzle for the break

The 8-puzzle rule from session 6: a tile can move from A to B **if** (1) A is adjacent to B **and** (2) B is blank.

Write it as a PDDL action schema `slide(?t ?from ?to)`. Which predicates do you need?

Part 4

Heuristics for free

Your session 6 heuristics, automatically

```
(:action slide
  :parameters (?t - tile
               ?from ?to - square)
  :precondition (and (at ?t ?from)
                     (adjacent ?from ?to) (1)
                     (blank ?to))         (2)
  :effect (and (at ?t ?to)
               (blank ?from)
               (not (at ?t ?from))
               (not (blank ?to))))
```

In session 6 *you* dropped conditions of the rule to get heuristics. In PDDL the rule *is* the precondition, so a planner can drop conditions by itself.

Which heuristic do you get?

1. Drop (2), keep (1)
2. Drop (1), keep (2)
3. Drop both

Your session 6 heuristics, automatically

```
(:action slide
  :parameters (?t - tile
               ?from ?to - square)
  :precondition (and (at ?t ?from)
                     (adjacent ?from ?to) (1)
                     (blank ?to))         (2)
  :effect (and (at ?t ?to)
               (blank ?from)
               (not (at ?t ?from))
               (not (blank ?to))))
```

In session 6 *you* dropped conditions of the rule to get heuristics. In PDDL the rule *is* the precondition, so a planner can drop conditions by itself.

Which heuristic do you get?

1. Drop (2), keep (1)
2. Drop (1), keep (2)
3. Drop both

(1) Manhattan distance h_2. (2) Gaschnig's h_3. (3) misplaced tiles h_1.

Key idea

Relaxing by editing the action schemas works for *every* domain. The question is which relaxation is easy to solve and still informative.

Relaxation 1: ignore preconditions

Drop *all* preconditions: every action is always applicable.

- Each step can make true any atoms it adds. How many steps to make all goal atoms true?
- If every action adds one goal atom: the **number of unsatisfied goal atoms**. That's the misplaced tiles heuristic.
- In general, some actions add several goal atoms: choosing the fewest actions is SET-COVER, NP-hard. Greedy set cover is fast, but may overestimate: no longer admissible.

Often too optimistic: it ignores everything you have to do *before* an action can be used.

Shopping

What does "ignore preconditions" give for the shopping trip? What does it miss?

Relaxation 1: ignore preconditions

Drop *all* preconditions: every action is always applicable.

- Each step can make true any atoms it adds. How many steps to make all goal atoms true?
- If every action adds one goal atom: the **number of unsatisfied goal atoms**. That's the misplaced tiles heuristic.
- In general, some actions add several goal atoms: choosing the fewest actions is Set-Cover, NP-hard. Greedy set cover is fast, but may overestimate: no longer admissible.

Often too optimistic: it ignores everything you have to do *before* an action can be used.

Shopping

What does "ignore preconditions" give for the shopping trip? What does it miss?

3: buy the three items, from home. It misses all the travelling: the true cost is 6.

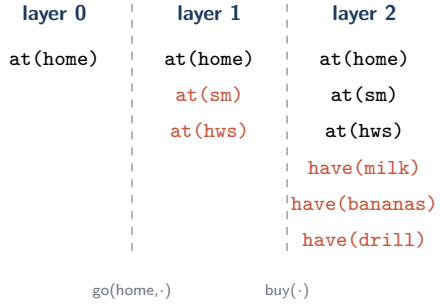
Relaxation 2: ignore delete lists

Keep the preconditions, drop all deletes (`not` in effects).

- Atoms are never lost: states only grow. Going to the supermarket doesn't take you away from home.
- The optimal cost of this relaxed problem is h^+. It is admissible, and usually very informative.
- But computing h^+ exactly is NP-hard too. So planners **approximate** it, in polynomial time.

Why it is easy to approximate

Without deletes, an action never hurts. Applying everything applicable, layer by layer, reaches every reachable atom within n layers.



The shopping trip without deletes (static `sells` atoms omitted). New atoms in red.

Two simple approximations: $h_{\max}$ and $h_{\mathbf{add}}$

Give every atom p a cost: 0 if p is in the current state, otherwise

$$cost(p) = \min_{a \text{ adds } p} \big(1 + cost(\textsc{Pre}(a))\big),$$

computed layer by layer until nothing changes. The cost of a *set* of atoms (a precondition, or the goal) is combined in one of two ways; h is the cost of the goal.

	cost of a set	admissible?	idea
$h_{\max}$	the **maximum** of its atoms' costs	yes	only the most expensive subgoal counts; all others are ignored
h_{add}	the **sum** of its atoms' costs	no	subgoals are treated as independent; steps they share are counted twice

$h_{\max} \leq h^+ \leq h_{\text{add}}$. Both take polynomial time.

With unit costs, an atom's $h_{\max}$ cost is simply the number of the first layer it appears in (slide 20).

But

Both only *combine numbers*. Neither of them looks at which actions the subgoals actually share.

A better approximation: h_{FF}

Idea: construct one concrete plan of the delete-relaxed problem, and count its actions. A step that several subgoals need is counted once.

1. Build the **relaxed planning graph** forwards: fact layers and action layers alternate, until all goal atoms appear.
2. Mark each goal atom as a subgoal at the first layer it appears in.
3. Go backwards, layer by layer. For each subgoal not in layer 0, choose *one* action from the action layer just before it that adds it. Its preconditions become subgoals (if they aren't already).
4. h_{FF} = the number of distinct actions chosen.

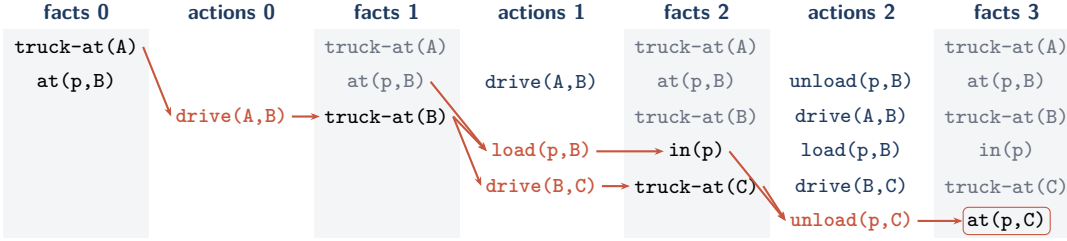
Properties

- The chosen actions form a relaxed plan, so $h^+ \leq h_{\mathsf{FF}}$.
- The choices are greedy: not always a shortest relaxed plan, so **not admissible**.
- Polynomial time, and in practice usually close to h^+.

The FF planner (Hoffmann & Nebel, 2001), greedy search with h_{FF}, made heuristic forward search the standard.

Example: a relaxed planning graph

A truck at `A` must bring package `p` from `B` to `C`; one-way roads $A \to B \to C$. Actions `drive`, `load`, `unload`, deletes ignored.



Build forwards. Action layer k: *all* actions whose preconditions are in fact layer k. Fact layer $k{+}1$: fact layer k plus their add effects (grey: already there). The layer where a fact first appears is its $h_{\max}$ cost.

Extract backwards (red): for each subgoal, one achiever in the layer before it first appears; its preconditions become subgoals. `truck-at(B)` serves two actions, but is achieved once. $h_{\max} = 3$ (the goal's layer), $h_{\text{add}} = 5$, $h_{\text{FF}} = 4 = h^*$.

Your turn: relaxed heuristics by hand

Pairs ▪ 10 min

The shopping trip, in the start state:

`at(home), sells(hws,drill), sells(sm,milk), sells(sm,bananas)`

Goal: `have(drill), have(milk), have(bananas), at(home)`

Actions: `go(?from,?to)` needs `at(?from)`; `buy(?i,?s)` needs `at(?s)` and `sells(?s,?i)`.

Your turn: Compute

1. Write down the cost of every atom (use the layers on slide 20).
2. Compute $h_{\max}$ and h_{add} for the goal.
3. Find a shortest plan of the delete-relaxed problem: that is h^+ (and here also h_{FF}).
4. Compare with the true cost $h^* = 6$. Which heuristic is closest? Which can overestimate?

Landmarks and LM-cut

A **landmark** is something that is true in *every* plan:

- a *fact landmark*: an atom that must become true at some point, e.g. `at(hws)`;
- an *action landmark*: an action every plan must use, e.g. `buy(drill,hws)`;
- a *disjunctive* action landmark: one of a set of actions, e.g. `go(home,hws)` or `go(sm,hws)`.

Counting the landmarks still to be reached gives a heuristic; the LAMA planner (Richter & Westphal, 2010) combines it with h^{FF}. **LM-cut** (Helmert & Domshlak, 2009) is a state-of-the-art landmark heuristic: it finds disjunctive action landmarks automatically and adds up their costs. Admissible, and the standard heuristic for *optimal* planning.

Shopping

Find three disjoint (disjunctive) action landmarks for the shopping trip that are not buy actions. What lower bound do the buys and these give together?

Landmarks and LM-cut

A **landmark** is something that is true in *every* plan:

- a *fact landmark*: an atom that must become true at some point, e.g. `at(hws)`;
- an *action landmark*: an action every plan must use, e.g. `buy(drill,hws)`;
- a *disjunctive* action landmark: one of a set of actions, e.g. `go(home,hws)` or `go(sm,hws)`.

Counting the landmarks still to be reached gives a heuristic; the LAMA planner (Richter & Westphal, 2010) combines it with h_{FF}. **LM-cut** (Helmert & Domshlak, 2009) is a state-of-the-art landmark heuristic: it finds disjunctive action landmarks automatically and adds up their costs. Admissible, and the standard heuristic for *optimal* planning.

Shopping

Find three disjoint (disjunctive) action landmarks for the shopping trip that are not buy actions. What lower bound do the buys and these give together?

Get to the SM (`go(·,sm)`), get to the HWS (`go(·,hws)`), get home (`go(·,home)`): with the three buys, $6 = h^*$. LM-cut gives 5 here: it looks for landmarks in the delete relaxation, which never needs the trip home.

Our numbers: 8 blocks

Two towers of 4 blocks into one tower of 8 (this file is in today's lab). Pyperplan in a web browser:

search	h	plan	expanded	time
A*	blind ($h = 0$)	16	66 594	2.1 s
A*	$h_{\max}$	16	4 426	6.4 s
A*	LM-cut	16	23	0.2 s
greedy	h_{FF}	16	46	0.1 s

Numbers vary a little from run to run: Pyperplan breaks ties differently each time.

Discuss with the room

- $h_{\max}$ expands 15 times fewer nodes than blind search, but takes 3 times longer. How can that be? Where have we seen this before?
- Which of these plans are guaranteed to be optimal?
- None of these heuristics knows anything about blocks. Where does their knowledge come from?

Lab: a planner in your browser

Pairs, one laptop ▪ 25 min

Your turn: Plan the Sussman anomaly, then 8 blocks

Files on Moodle: `blocksworld-domain.pddl`, `blocksworld-8.pddl`, `planner.ipynb`. Open `jupyterlite.github.io/demo/lab`, drag the three files into the file browser on the left, open the notebook, and run the first two cells (the first run takes about 20 seconds).

1. Write `sussman.pddl`, the problem file for the warm-up (it is on an earlier slide; try to write it without looking). Drag it in as well.
2. Solve it with `astar` and `blind`. Is the plan the one you found by hand?
3. Run `compare` on `blocksworld-8.pddl`. Do you get numbers like ours?
4. Make your own, bigger instance: add blocks `i` and `j`, or change the goal tower. Where does each configuration give up? Does greedy search with h_{FF} always find the shortest plan?

Problems? Read the error message: Pyperplan names the line or predicate it could not parse. Typical: a missing bracket, a predicate used with the wrong number of arguments, a type not declared.

Part 5

Other ways to plan

Planning as satisfiability (SAT)

Fix a plan length T. Make a propositional variable for every atom at every time $0..T$ and every action at every time $0..T-1$, e.g. $\texttt{on(A,B)}^3$, $\texttt{stack(A,B)}^2$. Clauses for:

- the initial state at time 0, the goal at time T;
- actions: $a^t \rightarrow \textsc{Pre}(a)^t$, and $a^t \rightarrow$ its effects at $t+1$;
- *frame axioms*: an atom only changes if some action changes it;
- at most one action per step (or only non-interfering ones).

A SAT solver finds a model: the true action variables are the plan. Unsatisfiable? Try $T+1$.

SATPLAN: Kautz & Selman (1992). It profits from every advance in SAT solvers.

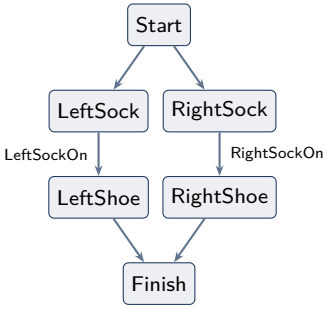
Looking ahead

Planning asks: can a goal state be reached? **Model checking** asks a similar question about a system model: can a *bad* state be reached, or does a property hold along all paths? We'll see how in the next session.

Question

Forward search or SAT: which one is better at proving that *no* plan of length $\leq T$ exists?

The old way: partial-order planning



One partial-order plan, six total orders.

Until the late 1990s, planners searched in the space of *plans*, not states:

- a plan is a set of steps, ordering constraints and **causal links** ("LeftSock achieves LeftSockOn for LeftShoe");
- repeatedly pick an open precondition and add a step (or reuse one) that achieves it;
- if a step could destroy a protected link (a **threat**), order it before or after the link.

Plans stay as unordered as possible: good for parallel execution and explanation. Without good heuristics, it lost out to forward search. Graphplan (Blum & Furst, 1995) was the bridge: the layers we built for $h_{\max}$ come from its planning graph.

Beyond classical planning

Classical planning assumes: one agent, a fully known start state, deterministic actions, no time, no numbers. R&N Chapter 11 continues with:

- **Hierarchical planning** (HTN): plan with high-level tasks ("build a house") and refine them into steps, using expert knowledge of how tasks decompose.
- **Nondeterminism and partial observability**: conditional plans, sensing actions, replanning during execution.
- **Time, schedules and resources**: actions with durations, limited cranes, berths, crews. Planning meets scheduling, and the methods from the first half of this course.
- **Uncertain outcomes with probabilities**: Markov decision processes, and learning them: *reinforcement learning*, the last topic of this course.

Planning competitions (IPC, since 1998) drive the field; PDDL has grown with them: numeric fluents, durative actions, preferences, probabilities.

Planning research today

An active research field, with its own conference: ICAPS (Int. Conference on Automated Planning and Scheduling), every year.

The International Planning Competition (IPC), every few years since 1998, held at ICAPS:

- planners run on new benchmark domains in PDDL (published afterwards);
- classical tracks for optimal, satisficing and fast ("agile") planning, and further tracks for numeric, temporal, hierarchical and probabilistic planning;
- most recent large edition 2023; a numeric track in 2026; the next classical track ends at ICAPS 2027.

The state of the art is open source

- **Fast Downward** (Basel, C++): implements all heuristics of today, from $h_{\max}$ to LM-cut, and the LAMA planner. Many recent IPC entries are built on it.
- **planutils**: installs many planners, from the IPC and elsewhere, with one command.

Pyperplan has the same ideas, written to be read, not to be fast. Your PDDL from Assignment 7 runs unchanged in Fast Downward.

Part 6

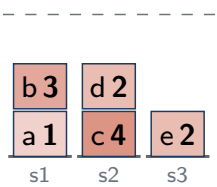
Assignment 7

Assignment 7: pre-marshalling in PDDL

The problem from Assignment 5, now as a planning domain. Ideas of the model (all in the skeleton file):

- every stack has a **base**; bases and containers are *places*;
- the crane picks up and puts down, as in Blocks World;
- **tiers** $t_0, t_1, \ldots$ for the height limit;
- groups are objects with a static order `(leq g1 g2)`;
- `(well-placed ?p)`: nothing blocking from p down. Goal: every container is well-placed.

Required: complete the two put-down actions and write the problem file for bay 1. **Optional**: run it, try more bays.



Bay 1: container name and **group**. Group 1 leaves first.

What we look for

- correct PDDL that a planner accepts
- a model you can *explain*: why is each precondition and effect there?

Assignment clinic

Pairs ▪ 12 min

Your turn: Start now, ask now

1. Open `premarshalling-domain.pddl`. Read `pick-up`: why does it delete `well-placed`?
2. Start TODO 1, `put-down-well`: when is a container well-placed after it is put on `?to`? Which parameters does the action need?
3. In bay 1: which containers are well-placed at the start?

Self-check questions

- Can your actions put a container on a full stack? On itself? Into the crane while it holds another one?
- Why is the goal not simply a list of `on` atoms?

Exit ticket

Alone, on a sticky note ▪ 3 min

Your turn: One-minute paper

1. One sentence: why can a planner compute a heuristic for a domain it has never seen?
2. The muddiest point today.

Next time: model checking. Reading: Baier & Katoen, *Principles of Model Checking*, Chapter 6 (Computation Tree Logic).

Key words today: STRIPS, PDDL, domain and problem file, action schema, precondition, add and delete list, grounding, progression, regression, relevant action, relaxation, ignore preconditions, delete relaxation, h^+, $h_{\max}$, h_{add}, h_{FF}, landmark, LM-cut, SAT planning, partial-order planning.